

"Code
Learn
Build
Repeat"

@abhi_techhub

Better
Developers
Brighter
Tomorrow 😊

Next.js

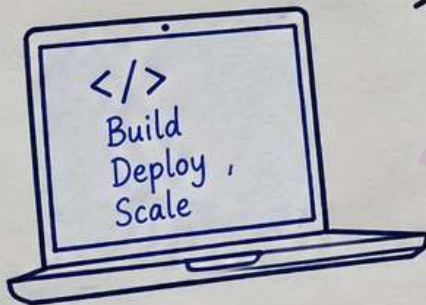
Complete Handbook

From Basics to Real-World Projects

Modern
React Framework
for
Real-World Apps



- ✓ Concepts
- ✓ Examples
- ✓ Best Practices
- ✓ Real-World Project
- ✓ Deployment
- ✓ Interview Questions



Write Once
Deploy Anywhere

Ideas
Projects
Skills
Career Growth

"Small
Steps
Big Results"



React
18/19



File-based
Routing



SSR
SSG
ISR
CSR



API
Routes



Database
Integration



Deployment
(Vercel)



Real-World
Project

Build Modern Web Apps
with Next.js



Good
Code
Better
Opportunities.

Prepared with ❤️ by
@abhi_techhub

Stay
Curious
Keep
Building 😊

Better
Developers
Build
Better
Products
...

Next.js

Complete Handbook.

@abhi_techhub

"The React Framework
for the Real World"

Build • Optimize • Deploy • Scale

1. What is Next.js ?

- Next.js is a React framework developed by Vercel that enables building full-stack web applications.
- It extends React with powerful features like server-side rendering, static site generation, API routes, and more.
- Helps in building fast, scalable and SEO-friendly web applications.

NEXT.JS

by  Vercel

"Next.js gives you the best developer experience with the performance and scalability of production."

2. React vs Next.js

Feature	React	Next.js
Type	Library	Framework
Rendering	Client-side (CSR)	CSR + SSR + SSG + ISR
Routing	Needs react-router	Built-in file-based routing
Backend/API	Not included	API routes / Route handlers
SEO	Limited (CSR)	SEO friendly (SSR/SSG)
Configuration	Manual setup	Zero configuration (create-next-app)
Use case	UI library for SPAs	Full-stack web apps (production ready)

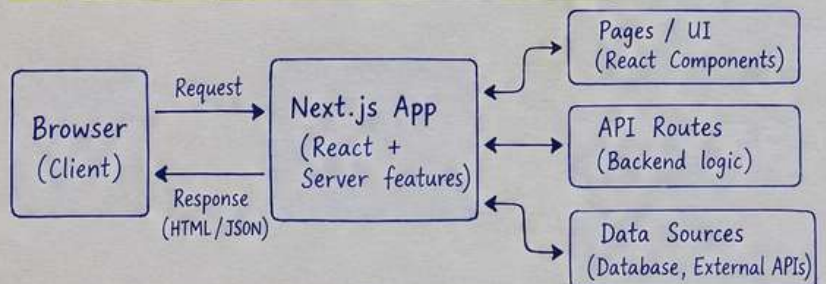
3. Why Next.js ?

- Server-side rendering (SSR) for better SEO
- Static generation (SSG) for performance
- Incremental Static Regeneration (ISR)
- Built-in routing, API routes
- Optimized image and font handling
- Great developer experience
- Used by top companies (Vercel, Netflix, Twitch, Nike, etc.)

4. Key Features

- File-based routing
- Server-side rendering (SSR)
- Static site generation (SSG)
- Incremental Static Regeneration (ISR)
- API routes / Route handlers
- React Server Components
- Built-in image optimization (next/image)
- Font optimization (next/font)
- Middleware
- Great DX with create-next-app
- Deployment on Vercel (easy)

5. Next.js Architecture (High Level)



6. Rendering Approaches in Next.js

- Client-Side Rendering (CSR) - UI rendered in browser
- Server-Side Rendering (SSR) - Renders on server per request
- Static Site Generation (SSG) - Pre-renders at build time
- Incremental Static Regeneration (ISR) - Updates static content over time

We'll
deep dive
in next
pages!



Fun Fact:

Next.js was created by Guillermo Rauch (the CEO of Vercel).

Page 2 : Project Setup & Folder Structure

"Little Setup
Big Possibilities"

1 Creating a Next.js App

- The easiest way to create a Next.js project is using `create-next-app`, which sets up everything for you (including latest best practices).

```
# Create a new Next.js app
npx create-next-app@latest my-app
cd my-app
npm run dev
```

You can also use additional options:

```
npx create-next-app@latest my-app \
  --typescript \
  --eslint \
  --tailwind \
  --src-dir \
  --app \
  --import-alias "@/*"
```

Creates a Next.js app with TypeScript, Tailwind CSS, app router and '@' alias for clean imports.

3 Understanding the app/ Folder

In Next.js (App Router), the `app/` directory contains all your routes, layouts and UI.

```

├── app/
│   ├── layout.tsx # Root layout (mandatory)
│   ├── page.tsx  # Home page (/)
│   ├── about/
│   │   └── page.tsx # /about route
│   ├── products/
│   │   └── [id]/
│   │       └── page.tsx # /products/[id]
│   ├── loading.tsx # Loading UI (optional)
│   ├── error.tsx  # Error UI (optional)
│   └── not-found.tsx # 404 page (optional)

```

Each folder inside `app/` becomes a route automatically based on its name.

5 public/ Folder

- Used to store static assets like images, fonts, icons, robots.txt, etc.
- Files inside `public/` can be accessed directly from the root URL.

Example:

- `public/logo.png` → accessible at `/logo.png`
- `public/robots.txt` → accessible at `/robots.txt`

2 Project Structure Overview

```

├── my-app/
│   ├── .next/      # Build output (auto-generated)
│   ├── app/        # App router (main code)
│   ├── public/     # Static files (images, icons, etc.)
│   ├── src/        # (optional) Source folder
│   │   ├── components/ # Reusable components
│   │   └── lib/        # Utilities / helpers
│   ├── .gitignore
│   ├── next.config.js # Next.js configuration
│   ├── package.json  # Project metadata & dependencies
│   ├── tsconfig.json # TypeScript configuration
│   └── README.md     # Project details

```

Note:

- `.next/` is created automatically after build.
- `app/` is the new way (App Router) introduced in Next.js 13+.
- `src/` is optional but keeps things organized.
- You can use JavaScript instead of TypeScript by not selecting `--typescript`.

4 Important Files

File	Purpose
<code>layout.tsx</code>	Defines shared UI (like navbar) for a route and its child routes.
<code>page.tsx</code>	Main UI for the route (required).
<code>loading.tsx</code>	Shown while the route is loading (supports React Suspense).
<code>error.tsx</code>	Shown when an error occurs in the route.
<code>not-found.tsx</code>	Custom 404 page for the route.
<code>global-error.tsx</code>	Handles uncaught errors in the entire app (optional).

6 src/ Folder (Optional)

- You can keep your code inside a `src/` folder for better organization.
- Not mandatory, but recommended for larger projects.

```

├── src/
│   ├── app/
│   ├── components/
│   ├── lib/
│   └── styles/

```

Functionality remains the same, just inside `src/` folder.



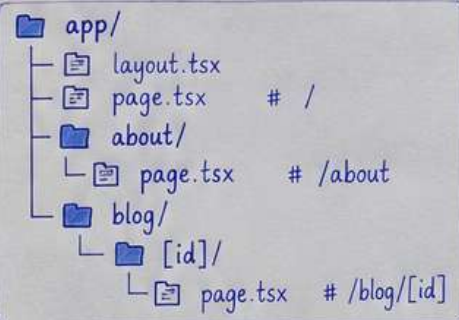
Pro Tip: Use a consistent folder structure and meaningful file names to keep your project scalable and easy to maintain.

"Good Routing
Makes a Great
User Experience"

1 File-based Routing

- Next.js uses file-based routing (App Router).
- Every folder inside `app/` becomes a route.
- No need to configure routes manually.
- Supports nested routes, dynamic routes, layouts and more.

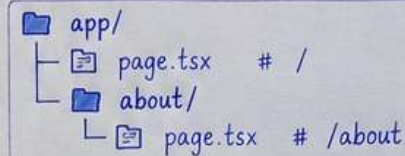
Example :



Each folder
= a route

2 Static Routes

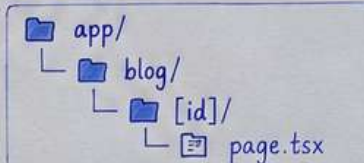
- A simple page inside `app/` folder is a static route.
- Example:



Route URL
matches the
folder name.

3 Dynamic Routes

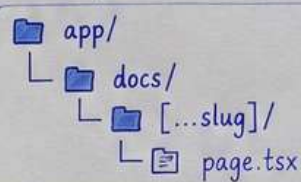
- Use square brackets `[ ]` to create dynamic routes.
- Example:



Route: `/blog/123`
id = 123
Access using:
params.id

4 Catch-all Routes

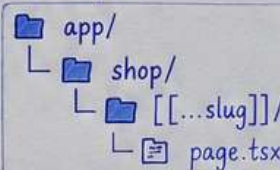
- Use `[...slug]` to match multiple segments.
- Example:



Matches:
`/docs/a`
`/docs/a/b`
`/docs/a/b/c`
All will
map to
`[...slug]`

5 Optional Catch-all Routes

- Use `[...slug]` to make the parameter optional.
- Example:



Matches:
`/shop` (slug = undefined)
`/shop/a` (slug = ["a"])
`/shop/a/b` (slug = ["a", "b"])

6 Navigation with <Link>

- Use Next.js `<Link>` component for client-side navigation (faster, no full page reload).

```

import Link from "next/Link";
export default function Navbar() {
  return (
    <nav>
      <Link href="/">Home</Link>
      <Link href="/about">About</Link>
      <Link href="/blog/1">Blog</Link>
    </nav>
  );
}
  
```

Link prefetches
the page in
background
⇒ Faster
navigation!

7 Using useRouter()

- `useRouter` hook is used for programmatic navigation.
- Example:

```

"use client"
import { useRouter } from "next/navigation";
export default function GoToAbout() {
  const router = useRouter();
  return (
    <button onClick={() => router.push("/about")}>
      Go to About
    </button>
  );
}
  
```

Common
methods:
• `push()`
• `replace()`
• `back()`
• `forward()`

8 redirect()

- Used to redirect users to another route.
- Can be used in Server Components, Server Actions or Route Handlers.

```

import { redirect } from "next/navigation";
export default function Page() {
  redirect("/login");
}
  
```

Stops rendering
the current page
and redirects
immediately.

Key Takeaways

- ✓ Next.js uses file-based routing (no manual config).
- ✓ Supports static, dynamic, catch-all and optional catch-all routes.
- ✓ Use `<Link>` for client-side navigation.
- ✓ Use `useRouter()` for programmatic navigation.
- ✓ Use `redirect()` for redirection (especially in server components).

1 What are Server Components ?

- Server Components are rendered on the server and sent to the client as HTML (no JS bundle).
- They are the **default** in Next.js (App Router).
- Can directly access backend resources (DB, APIs).
- Cannot use browser-only APIs (like `useState`, `useEffect`, `window`, etc.).

app/page.tsx (Server Component)

```
// This is a Server Component (default)
export default async function Home() {
  const res = await fetch('https://api.example.com/data');
  const data = await res.json();
  return (
    <div>
      <h1>Welcome to Next.js</h1>
      <p>Data: {data.name}</p>
    </div>
  );
}
```

No "use client"
= Server Component

2 What are Client Components ?

- Client Components are rendered on the client (browser).
- Used when you need interactivity (`useState`, `useEffect`, event handlers, etc.).
- Must include "use client" at the top of the file.
- They are included in the client-side JS bundle.

app/components/Counter.tsx (Client Component)

```
"use client";
import { useState } from "react";
export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

"use client"
= Client Component

3 Key Differences

Feature	Server Component	Client Component
Rendering	On Server	On Client (Browser)
Default	Yes (in App Router)	No (needs "use client")
Interactivity	No	Yes
<code>useState</code> / <code>useEffect</code>	Not allowed	Allowed
Access to DB / APIs	Yes (directly)	No (use API routes)
Included in JS bundle	No (only HTML)	Yes
Use case	Static content, data fetching, SEO	User interaction, forms, dynamic UI

4 When to Use What ?

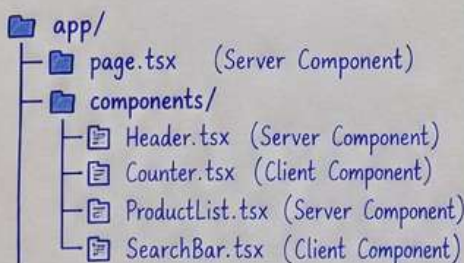
Use Server Components when:

- ✓ You are fetching data (DB, API)
- ✓ You don't need interactivity
- ✓ You want better performance & SEO
- ✓ Rendering static or mostly static content
- ✓ You want to reduce client-side JS

Use Client Components when:

- ✓ You need interactivity (buttons, forms, events)
- ✓ You are using hooks like `useState`, `useEffect`
- ✓ You need access to browser APIs (`window`, `localStorage`)
- ✓ You are building dynamic UI components (modals, tabs, etc.)

5 Component Tree Example



You can use Client Components inside Server Components, but **not** the other way around.

6 Important Points

- Server Components can import Client Components.
- Client Components cannot import Server Components.
- "use client" only needs to be in files that should run on the client.
- Keep your component tree minimal with client components.
- Use Server Components as much as possible for better performance.

? Common Interview Questions

- Why are Server Components better for performance?
- Can we use `useState` in a Server Component?
- Can a Server Component import a Client Component?
- What happens if we forget to add "use client"?



Pro Tip:

Use Server Components by default and add Client Components only when necessary.

1 Overview

- Next.js provides multiple ways to fetch data depending on your use case (Server, Client, Static, etc).
- In App Router, data fetching is done inside Server Components using `async/await`.
- You can also fetch data in Client Components using `useEffect` or libraries like SWR, React Query.
- Next.js automatically caches fetch requests to improve performance.



Key Idea:

Fetch data as close to where it's used, and by default, use Server Components for better performance and SEO.

2 Fetching Data in a Server Component

- Server Components can be `async`.
- Use the native `fetch()` API (built-in caching).
- No need to use `useEffect` or additional libraries.

app/page.tsx

```
export default async function Page() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts',
    { next: { revalidate: 60 } }); // cache for 60 seconds

  const posts = await res.json();

  return (
    <div>
      <h1>Posts</h1>
      <ul>
        {posts.slice(0, 5).map(post => (
          <li key={post.id}>{post.title}</li>
        ))}
      </ul>
    </div>
  );
}
```

- Runs on server
- Cached by default
- Use `revalidate` to control freshness

3 Fetching Data in a Client Component

- Use `useEffect` for basic fetching or libraries like SWR / React Query for better caching, revalidation and state management.

app/components/PostsClient.tsx

"use client"

```
import { useEffect, useState } from "react";

export default function PostsClient() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(res => res.json())
      .then(data => setPosts(data.slice(0, 5)));
  }, []);

  return (
    <div>
      <h2>Posts (Client)</h2>
      <ul>
        {posts.map(post => (
          <li key={post.id}>{post.title}</li>
        ))}
      </ul>
    </div>
  );
}
```

- Runs in browser
- Use for interactions and real-time data

4 Request Memoization & Caching

- Next.js automatically memoizes `fetch()` requests with the same URL and options.
- Prevents duplicate network requests.
- You can control caching behavior using the `next` option.

Option	Description
<code>cache: 'force-cache'</code>	Cache the response (default).
<code>cache: 'no-store'</code>	Do not cache (always fetch fresh).
<code>next: { revalidate: n }</code>	Revalidate after n seconds (ISR).
<code>next: { tags: ['tag'] }</code>	Assign tags for manual revalidation.

Example: No Caching (always fresh data)

```
const res = await fetch("https://api.example.com/data", {
  cache: 'no-store'
});
```

5 Parallel vs Sequential Fetching

- Use `Promise.all()` to fetch multiple data sources in parallel for better performance.

Sequential Fetching (Slower)

```
const user = await fetchUser();
const posts = await fetchPosts();
const comments = await fetchComments();
```

Parallel Fetching (Faster)

```
const [user, posts, comments] =
  await Promise.all([
    fetchUser(),
    fetchPosts(),
    fetchComments()
  ]);
```

6 Key Takeaways

- ✓ Prefer Server Components for data fetching.
- ✓ Use caching and revalidation wisely.
- ✓ Use Client Components only when you need interactivity.
- ✓ Use `Promise.all()` for parallel fetching.
- ✓ Leverage Next.js caching to reduce load time and API calls.

Data drives the UI → Fetch it right! 😊

Page 6 : Rendering Strategies

"Choose the right rendering strategy for a faster web"

1 Why Rendering Strategy Matters ?

- Different pages have different requirements.
- Choosing the right rendering strategy helps in:
 - Better performance
 - Improved SEO
 - Lower server cost
 - Better user experience

Same Next.js app can have both static and dynamic pages!

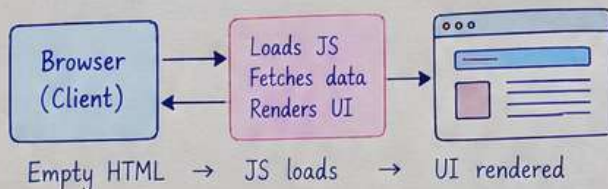
2 Types of Rendering in Next.js

- Client-Side Rendering (CSR)
- Server-Side Rendering (SSR)
- Static Site Generation (SSG)
- Incremental Static Regeneration (ISR)
- Dynamic Rendering (on-demand)

Next.js gives you the flexibility to choose the best strategy per page!

3 Client-Side Rendering (CSR)

- Page is rendered in the browser using JavaScript.
- HTML is minimal, data is fetched on client.
- Slower initial load but fast navigation after load.
- Good for highly interactive applications (dashboards, user-specific data).



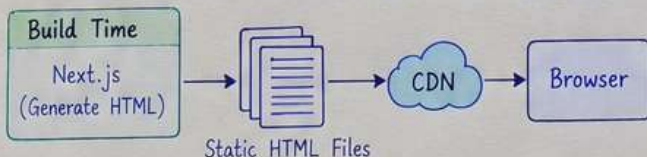
4 Server-Side Rendering (SSR)

- Page is rendered on the server for every request.
- Returns fully rendered HTML to the browser.
- Better SEO and faster first paint.
- Suitable for dynamic content (e.g., user-specific, frequently changing data).



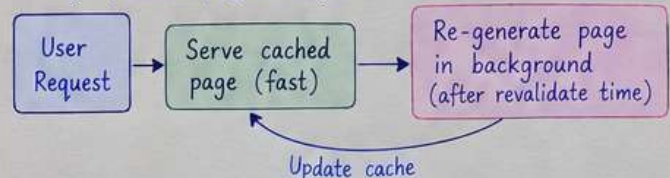
5 Static Site Generation (SSG)

- Pages are pre-rendered at build time.
- Generates static HTML files.
- Very fast, great for SEO.
- Best for static content (blogs, marketing pages).



6 Incremental Static Regeneration (ISR)

- Allows you to update static content after build time.
- Pages are re-generated in the background.
- Combines benefits of SSG (speed) and dynamic content.
- Useful for content that changes occasionally (e.g., product pages, blogs).



7 Comparison Table

Feature	CSR	SSR	SSG	ISR
When rendered	In Browser	On each request	At build time	Build + Revalidate
Initial Load	Slower	Faster	Fastest	Fast
SEO	Poor	Good	Excellent	Excellent
Best for	Interactive apps	Dynamic content	Static content (blogs, docs)	Static + updated content
Server load	Low	High	Low	Low
Examples	Dashboard	E-commerce	Blog/Marketing	News, Products

8 Code Example

```
// ISR example (revalidate every 60 seconds)
export const revalidate = 60;

export default async function Page() {
  const res = await fetch(
    'https://api.example.com/products',
    { next: { revalidate: 60 } }
  );
  const products = await res.json();
  return (
    <div>
      {products.map(p => <ProductCard key={p.id} product={p} />)}
    </div>
  );
}
```



Key Takeaways :

- Use the right rendering strategy based on your use case.
- SSG/ISR for content pages, SSR for dynamic pages, CSR for interactive apps.
- Next.js gives the flexibility to use different strategies in the same app.

Render Smart, Scale Faster! 😊

1 Overview

- Next.js has a powerful caching system to improve performance and reduce server load.
- It automatically caches fetch requests, page output, and even router data.
- You can control caching behaviour using options like `revalidate`, `tags` and `dynamic rendering`.
- Understanding caching is important for building fast and scalable applications.

Key Caches in Next.js:

- Data Cache → caches `fetch()` requests
- Full Route Cache → caches rendered page output
- Router Cache → caches client-side navigations
- Request Memoization → dedupes identical requests

3 Data Cache

- Next.js automatically caches `fetch()` requests on server.
- By default, `fetch()` is cached (`force-cache`).
- You can control caching using `cache` option.

```
// Cached (default)
const res = await fetch("https://api.example.com/data");

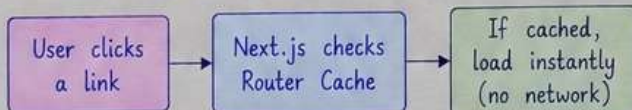
// No cache
const res = await fetch("https://api.example.com/data", {
  cache: 'no-store'
});

// Revalidate after 60 seconds
const res = await fetch("https://api.example.com/data", {
  next: { revalidate: 60 }
});
```

Data Cache
persists across
requests and
deployments
(on Vercel).

5 Router Cache

- Caches route segments in the browser for faster client-side navigation.
- Works automatically with the App Router.
- Reduces network requests when navigating between pages.



7 Revalidation

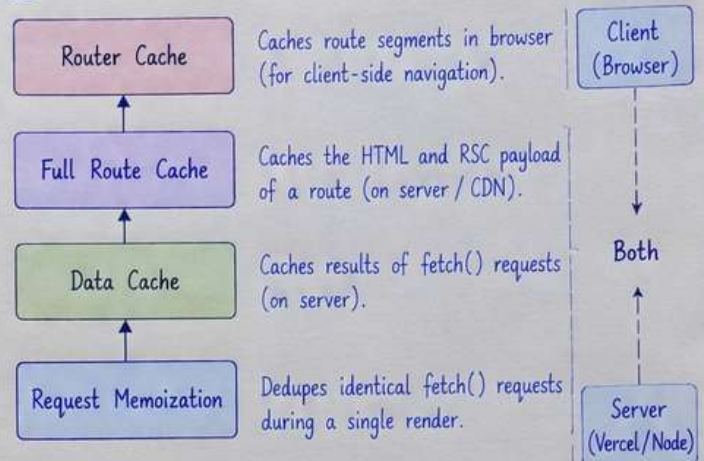
- You can manually revalidate cached content using:
 - `revalidatePath()` → revalidate a specific path
 - `revalidateTag()` → revalidate all fetches with a tag
- Useful for dynamic updates (e.g., after a form submission).

```
import { revalidatePath, revalidateTag } from "next/cache";
export async function updatePost() {
  // after updating data
  revalidatePath("/blog");
  revalidateTag("posts");
}
```

Tagging fetch requests:

```
await fetch("https://api.example.com/posts", {
  next: { tags: ['posts'] }
});
```

2 Next.js Caching Layers



4 Full Route Cache

- Caches the rendered output (HTML + RSC payload).
- Used for static routes and ISR.
- Helps in serving pages quickly from CDN.
- Automatically invalidated when you revalidate.

Example (ISR):

```
export const revalidate = 60;
export default async function Page() {
  const data = await fetch("https://api.example.com/posts", {
    next: { revalidate: 60 }
  });
  return <div>Posts</div>
}
```

Page will be cached
and re-generated
in background
after 60 seconds.

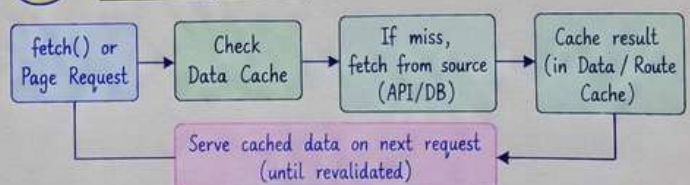
6 Request Memoization

- Dedupes identical `fetch()` requests within the same render pass.
- If the same URL is fetched multiple times, Next.js will only make one network request.

```
// Both calls will be deduped
const data1 = await fetch("https://api.example.com/user");
const data2 = await fetch("https://api.example.com/user");
```

Useful in
layouts and
nested components.

8 Summary Flow



9 Key Takeaways

- Next.js has multiple caching layers (Data, Route, Router).
- Use `revalidate` for time-based updates (ISR).
- Use `revalidatePath()` or `revalidateTag()` for manual revalidation.
- Caching improves performance and reduces server load.
- Understand when to use cache: "no-store" vs "force-cache".

Page 8 : Layouts, Loading & Error Handling

1 Layouts in Next.js

- Layouts allow you to share UI between multiple pages.
- Defined using `layout.tsx` inside a route segment.
- Layouts persist across route changes (do not re-render).
- You can have nested layouts.
- Commonly used for Navbar, Sidebar, Footer, Providers etc.

app/layout.tsx (Root Layout)

```
export default function RootLayout({
  children,
}: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body>
        <Navbar />
        {children}
        <Footer />
      </body>
    </html>
  );
}
```

This layout will be shared across all pages in the app.

Use nested layouts for route-specific UI.

2 Nested Layouts

- Each folder can have its own `layout.tsx`.
- Layouts can be nested to create complex UI structures.
- Useful for grouping related routes.

```

app/
├── layout.tsx # Root layout
├── dashboard/
│   ├── layout.tsx # Dashboard layout
│   └── page.tsx
└── settings/
    ├── layout.tsx # Settings layout
    └── page.tsx
  
```

Child layout is rendered inside the parent layout using `{children}`

3 loading.tsx (Loading UI)

- Special file to show loading state while a page or component is being rendered (especially with server components).
- Automatically shown during data fetching.
- Used with React Suspense under the hood.

app/products/loading.tsx

```
export default function Loading() {
  return (
    <div className="p-4">
      <p>Loading products...</p>
    </div>
  );
}
```

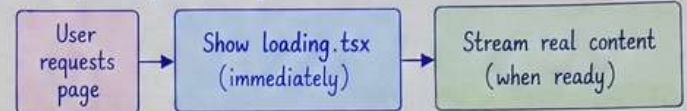
This will be shown automatically while the `page.tsx` is loading (fetching data, etc.)

8 Key Takeaways

- ☑ Use layouts to share common UI.
- ☑ Use nested layouts for better structure.
- ☑ Use `loading.tsx` for better user experience (streaming).
- ☑ Handle errors gracefully with `error.tsx` and `global-error.tsx`.
- ☑ Use `not-found.tsx` for 404 pages.

4 Streaming with Suspense

- Next.js uses React Suspense to stream UI from the server.
- You can show a loading UI immediately and stream the actual content when it's ready.
- Improves perceived performance.



5 error.tsx (Error Handling)

- Special file to handle runtime errors in a route segment.
- Shows a fallback UI when an error occurs while rendering the page or its children.
- Works with React Error Boundaries.

app/products/error.tsx

```
"use client";

export default function Error({
  error,
  reset,
}: {
  error: Error;
  reset: () => void;
}) {
  return (
    <div className="p-4">
      <h2>Something went wrong!</h2>
      <p>{error.message}</p>
      <button onClick={reset}>Try Again</button>
    </div>
  );
}
```

"use client" is required for `error.tsx` because it uses client-side interactivity (e.g. `reset`).

`reset()` retries the rendering of the segment.

6 not-found.tsx (404 Page)

- Special file to handle 404 - when a route is not found.
- Automatically shown when you call `notFound()` or when a route does not exist.

app/not-found.tsx

```
export default function NotFound() {
  return (
    <div className="p-8 text-center">
      <h1 className="text-2xl">404 - Page Not Found</h1>
      <p>The page you are looking for does not exist.</p>
    </div>
  );
}
```

You can also use `notFound()` from "next/navigation" inside a component or server action.

7 global-error.tsx

- Handles unexpected errors in the entire application.
- Used for critical failures (e.g. root layout errors).
- Acts as a global fallback UI.

1 Overview

- Next.js allows you to create backend APIs using Route Handlers (in App Router).
- You can handle GET, POST, PUT, DELETE requests.
- Useful for small to medium backends, webhooks, or proxying to external services.
- No need for a separate backend server in many cases.
- Built on top of the Web Fetch API (Request/Response).

2 File Structure

```

app/
├── api/
│   ├── users/
│   │   └── route.ts # Handles /api/users
│   └── products/
│       └── route.ts # Handles /api/products

```

Each folder inside /api represents an endpoint.

3 Basic GET Example

```

app/api/hello/route.ts
export async function GET() {
  return Response.json({
    message: "Hello from Next.js API",
  });
}

```

Uses Response.json() to return JSON response.

4 Handling Multiple Methods

```

app/api/users/route.ts
export async function GET() {
  return Response.json({ users: ["Abhi", "Rohan"] });
}

export async function POST(request: Request) {
  const body = await request.json();
  const { name, email } = body;
  // save to DB
  return Response.json({ message: "User created", user: { name, email } }, { status: 201 });
}

```

Each HTTP method is exported as a separate function.

5 Request & Response Objects

- Next.js uses the Web Fetch API standard.
- Request: contains URL, headers, body, etc.
- Response: used to return data to the client.

Common Request Methods

- request.json() → Parse JSON body
- request.headers → Read headers
- request.url → Get full URL
- request.cookies → Read cookies

Common Response Helpers

- Response.json(data)
- new Response(data)
- Response.redirect(url)
- Set status code
- Set headers

6 Dynamic API Routes

```

app/api/users/[id]/route.ts
export async function GET(
  request: Request,
  { params }: { params: { id: string } }
) {
  const { id } = params;
  return Response.json({ userId: id });
}

```

Use [id] for dynamic routes. You can access route params from the second argument.

7 Query Parameters

```

app/api/aserch/route.ts
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const query = searchParams.get("q");
  return Response.json({ query });
}

```

Use searchParams to get query parameters.

8 Headers & Cookies

```

export async function GET(request: Request) {
  const token = request.headers.get("authorization");
  const cookie = request.cookies.get("session-id");

  return Response.json({
    token,
    cookie: cookie?.value,
  });
}

```

You can read headers and cookies directly from the request object.

9 Returning Different Responses

```

return new Response("Not Found", { status: 404 });
return Response.json({ message: "Created" }, { status: 201 });
return Response.redirect("/dashboard", 302);

```

You can return custom responses, status codes and redirects.

10 Use Cases

- Building small APIs (auth, forms, etc.)
- Proxying requests to external services
- Webhooks (e.g., Stripe, Razorpay)
- Handling form submissions
- Reading/setting cookies
- Feature flags, A/B testing
- Internal tools and admin panels

Things to Keep in Mind

- Not ideal for large, complex backends.
- Use proper validation (e.g., Zod).
- Handle errors properly.
- Use environment variables for secrets.
- Consider rate limiting for public APIs.

11 Key Takeaways

- Route Handlers make it easy to build APIs in Next.js.
- Each HTTP method is a separate exported function.
- Use Request/Response objects from the Fetch API.
- Supports dynamic routes, query params, headers and cookies.
- Great for small backends, webhooks and proxying.
- Follow best practices for validation and error handling.

Page 10 : Authentication & Middleware

1 Why Authentication?

- Protects user data and restricted routes.
- Provides personalized experiences.
- Required for real-world applications (dashboards, e-commerce, admin panels, etc.).
- Can be implemented using NextAuth.js, JWT, or custom authentication.

Common Use Cases:

- User Login / Signup
- Social Login (Google, GitHub, etc.)
- Session Management
- Role-based Access (Admin/User)

Next.js works well with NextAuth.js for easy authentication.

2 NextAuth.js (Popular Choice)

- Open source authentication library for Next.js.
- Supports multiple providers (Google, GitHub, Credentials, etc.).
- Handles sessions, JWT, and callbacks easily.
- Works with both Next.js (Pages Router & App Router).

Installation:

```
npm install next-auth
```

Basic Configuration:

app/api/auth/[...nextauth]/route.ts

```
import NextAuth from "next-auth";
import GoogleProvider from "next-auth/providers/google";

export const { GET, POST } = NextAuth({
  providers: [
    GoogleProvider({
      clientId: process.env.GOOGLE_CLIENT_ID!,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
    }),
  ],
  session: { strategy: "jwt" },
});
```

Configure your providers (Google, GitHub, Credentials, etc.)

3 Using Session in Components

```
import { useSession, signIn, signOut } from "next-auth/react";
export default function Profile() {
  const { data: session, status } = useSession();

  if (status === "loading") return <p>Loading...</p>;
  if (!session) return (
    <button onClick={() => signIn()}>Login</button>
  );

  return (
    <div>
      <p>Welcome {session.user?.name}</p>
      <button onClick={() => signOut()}>Logout</button>
    </div>
  );
}
```

useSession() gives you current user session info.

4 Protecting Routes (Middleware)

- Use middleware.ts to protect specific routes.
- Runs before a request is completed.
- Useful for redirecting unauthenticated users.

middleware.ts (in root)

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
  const token = request.cookies.get("next-auth.session-token");

  if (!token && request.nextUrl.pathname.startsWith("/dashboard")) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/dashboard/:path*"],
};
```

Redirects unauthenticated users to /login

5 Role Based Access Control (RBAC)

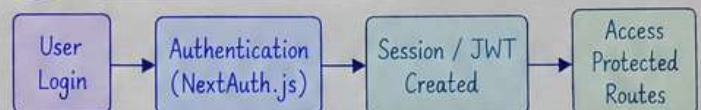
- Assign roles to users (e.g., admin, user, editor).
- Store role in JWT or database.
- Check role inside middleware or in components.

Example: Check Admin Role

```
const session = await getServerSession(authOptions);
if (session?.user?.role !== "admin") {
  return NextResponse.redirect(new URL("/unauthorized", request.url));
}
```

You can store role in JWT or in database during login.

6 Authentication Flow



7 Best Practices

- ✓ Use NextAuth.js for most use cases.
- ✓ Store sensitive keys in environment variables.
- ✓ Use middleware for route protection.
- ✓ Implement role-based access for admin routes.
- ✓ Keep session strategy as "jwt" for scalability.
- ✓ Handle errors and expired sessions gracefully.
- ✓ Test authentication flow thoroughly.

8 Key Takeaways

- ✓ Authentication is essential for secure applications.
- ✓ NextAuth.js makes it simple to implement.
- ✓ Middleware helps protect routes.
- ✓ Use roles/permissions for fine-grained access control.
- ✓ Follow best practices for security.

1 Overview

- Next.js provides multiple ways to fetch data.
- You can fetch data on Server (SSR/SSG/ISR) or on Client (CSR).
- Choice depends on use case: performance, SEO, real-time data, etc.
- Data fetching is done using `fetch()` or external libraries like `Axios`.

2 Data Fetching Methods

Method	When it Runs	Use Case	Key Function
SSR	On every request (server)	Dynamic data, SEO important	<code>async function Page()</code>
SSG	At build time	Static content (blogs, docs)	<code>generateStaticParams</code> / <code>fetch()</code>
ISR	At build time + revalidate	Content that changes	<code>revalidate</code> option
CSR	In browser (client)	User-specific data, real-time	<code>useEffect()</code> , <code>SWR</code> , <code>React Query</code>

3 Server-Side Data Fetching (SSR)

- Data is fetched on each request.
- Always fresh data.
- Useful for dynamic content (e.g., product details, dashboard).

app/products/[id]/page.tsx

```
export default async function Page({ params }: {
  params: { id: string }
}) {
  const res = await fetch("https://api.example.com/products/${params.id}");
  const product = await res.json();

  return (
    <div>
      <h1>{product.name}</h1>
      <p>{product.description}</p>
    </div>
  );
}
```

This page is rendered on server for every request.

4 Static Site Generation (SSG)

- Data is fetched at build time.
- Best for static content (blogs, marketing pages).
- Uses `generateStaticParams()` for dynamic routes.

app/blog/[slug]/page.tsx

```
export async function generateStaticParams() {
  const res = await fetch("https://api.example.com/blogs");
  const blogs = await res.json();
  return blogs.map((b: any) => ({ slug: b.slug }));
}

export default async function Page({ params }: { params: { slug: string } }) {
  const res = await fetch("https://api.example.com/blogs/${params.slug}");
  const blog = await res.json();
  return <div>{blog.title}</div>;
}
```

Pages are pre-rendered at build time.

5 Incremental Static Regeneration (ISR)

- Allows you to update static content after build time.
- Page is re-generated in the background.
- Useful for content that changes occasionally.

app/news/[id]/page.tsx

```
export const revalidate = 60; // revalidate every 60s

export default async function Page({ params }: {
  params: { id: string }
}) {
  const res = await fetch("https://api.example.com/news/${params.id}");
  const news = await res.json();
  return <div>{news.title}</div>;
}
```

Page is regenerated in background after 60 seconds.

6 Client-Side Data Fetching (CSR)

- Data is fetched in the browser using `useEffect()`.
- Useful for user-specific data or real-time updates.
- Can also use `SWR` or `React Query` for better caching.

app/profile/page.tsx

```
"use client";
import { useEffect, useState } from "react";

export default function Profile() {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch("/api/profile")
      .then(res => res.json())
      .then(setData);
  }, []);

  return <div>{data ? <p>{data.name}</p> : <p>Loading...</p>}</div>;
}
```

Data is fetched in browser after component mounts.

7 Comparing Rendering Strategies

Feature	SSR	SSG	ISR	CSR
Data Freshness	Always fresh	Static	Periodic update	Real-time
Build Time	No	Yes	Yes	No
Server Load	High	Low	Medium	Low
SEO	Good	Excellent	Excellent	Poor
Best For	Dynamic pages	Static pages	Changing content	User-specific / interactive

8 Key Takeaways

- ✓ Choose the right data fetching strategy based on use case.
- ✓ SSR for dynamic and SEO content.
- ✓ SSG for static content.
- ✓ ISR for content that changes occasionally.
- ✓ CSR for user-specific or real-time data.
- ✓ Next.js gives flexibility to mix different strategies in the same app.
- ✓ Use caching libraries (`SWR`, `React Query`) for better performance.

1 Why Optimization ?

- Faster load times → better user experience.
- Improves Core Web Vitals (LCP, FID, CLS).
- Helps in SEO ranking
- Reduces server costs
- Essential for production-ready apps

A faster app
= happier
users
= higher
conversions

2 Image Optimization

- Use next/image for automatic optimization.
- Supports lazy loading, resizing and modern formats (WebP, AVIF).
- Reduces page size significantly.

import Image from "next/image";

```
export default function ProductCard() {
  return (
    <div>
      <Image
        src="/images/shoe.jpg"
        alt="Shoe"
        width={500}
        height={300}
        priority={false} // lazy load by default
      />
    </div>
  );
}
```

Benefits:

- Automatic image optimization
- Lazy loading by default
- Serves modern formats (WebP)

3 Code Splitting & Dynamic Imports

- Next.js automatically code splits each page.
- Use dynamic imports for heavy components.

import dynamic from "next/dynamic";

```
const HeavyComponent = dynamic(
  () => import("../components/HeavyComponent"),
  { loading: () => <p>Loading...</p> }
);

export default function Page() {
  return (
    <div>
      <h1>Dashboard</h1>
      <HeavyComponent />
    </div>
  );
}
```

Useful for
large libraries
(e.g., charts,
editors, maps)
that are not
needed
immediately.

4 Caching Strategies

Type	Description	Use Case	Example
Browser Cache	Stores static assets in browser	Images, JS, CSS	Cache-Control headers
Data Cache	Caches fetch requests (SSR/SSG)	API responses	fetch(..., { next: { revalidate } })
ISR Cache	Regenerates page after time	Blog posts, products	export const revalidate = 60
CDN Cache	Edge caching (globally)	Static + ISR pages	Vercel CDN (automatic)

5 SEO in Next.js

- Use next/head or metadata API for meta tags.
- Supports dynamic meta tags per page.
- Generate Open Graph and Twitter cards.
- Use semantic HTML and proper heading structure.

import Head from "next/head";

```
export default function Page() {
  return (
    <>
      <Head>
        <title>Product | MyStore</title>
        <meta name="description" content="Buy shoes online" />
        <meta property="og:title" content="MyStore" />
        <meta property="og:description" content="Best shoes" />
        <meta property="og:image" content="/og-image.jpg" />
      </Head>
      <h1>Our Products</h1>
    </>
  );
}
```

You can also use
the new Metadata
API in app directory
(layout.tsx / page.tsx)

6 Performance Monitoring

- Use tools to analyze performance.
- Measure Core Web Vitals (LCP, FID, CLS).
- Identify and fix bottlenecks (large bundles, unused code).

Tools:

- Google Lighthouse
- Vercel Analytics
- Web Vitals Extension
- Chrome DevTools (Performance tab)

Target Core Web Vitals:
LCP < 2.5s
FID < 100ms
CLS < 0.1

7 Production Best Practices

- ☑ Use environment variables (.env) for secrets.
- ☑ Enable compression (automatic on Vercel).
- ☑ Optimize images and assets.
- ☑ Use dynamic imports for large modules.
- ☑ Implement proper error handling (error.tsx, not-found.tsx).
- ☑ Use middleware for redirects and auth.
- ☑ Follow SEO best practices.
- ☑ Test on production build: npm run build && npm start.

8 Key Takeaways

- ☑ Optimization improves speed, SEO and user experience.
- ☑ Use Next.js built-in features like next/image, dynamic imports, and caching.
- ☑ Always monitor performance and fix issues.
- ☑ Follow best practices for production deployment.
- ☑ A well-optimized Next.js app scales better and performs reliably.

1 Deployment Options

- Next.js apps can be deployed on multiple platforms.
- Vercel is the recommended platform (by Next.js team).
- You can also deploy on Netlify, AWS, Docker, or any VPS.
- Supports both static and dynamic deployments.

Platform	Pros	Use Case
Vercel	Easy, optimized, CI/CD, Edge network	Most common (Recommended)
Netlify	Simple, good for static sites	Small to medium apps
AWS (EC2/S3)	Full control, scalable	Production, custom infra
Docker	Portable, consistent environment	Enterprise / Self-hosted
Other (Render, Railway, etc.)	Simple setup	Hobby / small projects

2 Deploying on Vercel (Easiest)

- Push your code to GitHub.
- Go to vercel.com and import your repository.
- Configure build settings (auto-detected).
- Click Deploy → your app is live 🚀.

vercel.json (optional configuration)

```
{
  "buildCommand": "npm run build",
  "outputDirectory": ".next",
  "installCommand": "npm install",
  "framework": "next"
}
```

Vercel automatically detects Next.js and applies optimal settings.

3 Environment Variables

- Used to store sensitive data (API keys, DB URLs, etc.).
- In Next.js, use `.env.local` for local development.
- Use Vercel dashboard to set environment variables in production.
- Access variables using `process.env`.

.env.local

```
NEXT_PUBLIC_API_URL=https://api.example.com
DATABASE_URL=postgres://user:pass@localhost:5432/db
NEXTAUTH_SECRET=your-secret-key
NEXTAUTH_URL=http://localhost:3000
```

Variables starting with `NEXT_PUBLIC_` are exposed to the browser. Others are server-only.

Using in Code

```
const apiUrl = process.env.NEXT_PUBLIC_API_URL;
const dbUrl = process.env.DATABASE_URL;
```

4 Build Process

- Next.js compiles your code, optimizes assets, and generates the `.next` folder.
- For production, always use `npm run build` followed by `npm start` (or use platform-specific commands like `vercel`).

Build Commands

```
npm run build # creates optimized production build
npm run start # starts the production server
npm run dev # runs in development mode
```

`.next` folder contains optimized production build files.

5 CI/CD with GitHub Actions

- Automate testing, building and deployment.
- Commonly used with Vercel, Docker, or AWS.
- Runs on every push or pull request.

.github/workflows/deploy.yml (Example)

```
name: Deploy Next.js App
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm install
      - run: npm run build
      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }
          vercel-org-id: ${ secrets.ORG_ID }
          vercel-project-id: ${ secrets.PROJECT_ID }
```

Add your secrets in GitHub repo settings (VERCEL_TOKEN, ORG_ID, etc.)

6 Common Deployment Issues

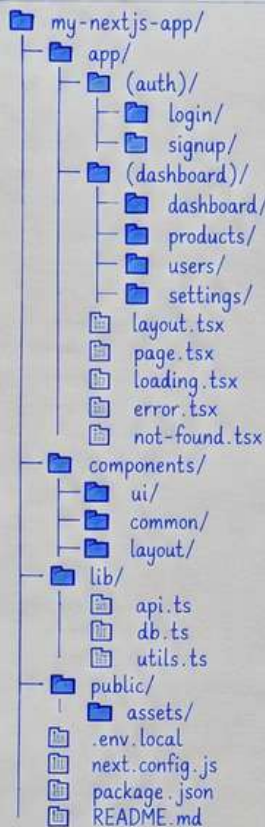
Issue	Cause	Solution
Build fails	Node version mismatch	Use correct Node version (e.g. 18/20)
Env variable not found	Not set in platform	Add env variables in Vercel/host dashboard
Images not loading	Missing domains config	Add domain in <code>next.config.js</code> (images)
API routes failing	Wrong base URL	Use relative paths or correct env variables
Slow performance	No caching/large bundles	Enable caching, optimize images, code splitting

7 Key Takeaways

- ✓ Vercel is the easiest and best platform for Next.js.
- ✓ Use environment variables for sensitive data.
- ✓ Set up CI/CD for automatic deployment.
- ✓ Monitor builds and fix errors early.
- ✓ Follow best practices for smooth and reliable deployment.

Page 14 : Real World Project Structure & Advanced Concepts

1 Real World Project Structure



Use route groups () to organize routes without affecting the URL structure.

Keep it Modular :

- Separate components
- Use reusable UI components
- Keep API logic in lib/ or services/
- Use environment variables
- Follow feature-based structure

4 Server Components vs Client Components

Server Components (Default)	Client Components ("use client")
• Rendered on server • Faster performance • Can fetch data directly • Not interactive (no event handlers) • Ideal for static or data-heavy pages	• Rendered on client (browser) • Can use state and hooks • Interactive (onClick, useState, etc.) • Needed for UI interactivity • Use "use client" at the top of the file

Example:

```
// Server Component (default)
export default async function Page() {
  const data = await getData();
  return <div>{data.title}</div>;
}
```

```
// Client Component
"use client";
import { useState } from "react";
export default function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() =>
    setCount(count + 1)}>{count}</button>;
}
```

Use Server Components by default and Client Components only when interactivity is required.

5 Error Handling

- Use error.tsx for route-level error handling.
- Use try-catch for API calls.
- Show user-friendly error messages.
- Log errors using monitoring tools (e.g., Sentry).

app/error.tsx

```
"use client";
export default function Error({
  error,
  reset,
}: {
  error: Error;
  reset: () => void;
}) {
  return (
    <div className="p-10 text-center">
      <h2 className="text-xl font-bold">Something went wrong!</h2>
      <p className="text-gray-600">{error.message}</p>
      <button
        onClick={reset}
        className="mt-4 px-4 py-2 bg-blue-500 text-white"
      >Try Again</button>
    </div>
  );
}
```

This will catch runtime errors in a route and show a fallback UI.

2 API Integration in Real Projects

- Use fetch() or Axios to call external APIs.
- Create a separate file for API calls (e.g., lib/api.ts).
- Use environment variables for API base URLs.
- Handle loading, error and success states.
- Use React Query or SWR for better data fetching and caching.

lib/api.ts

```
export const API_URL = process.env.NEXT_PUBLIC_API_URL;

export async function getProducts() {
  const res = await fetch(`${API_URL}/products`);
  if (!res.ok) throw new Error("Failed to fetch");
  return res.json();
}
```

Keep API logic in a separate file for reusability and ease of maintenance.

3 State Management Options

Option	Use Case	Pros	Example
useState	Local UI state	Simple	const [count, setCount] = useState(0)
useContext	Global state (small apps)	Built-in	Theme, Auth
Zustand	Global state (medium/large)	Lightweight, simple API	create(...)
Redux Toolkit	Large, complex apps	Powerful, scalable	createSlice(...)
React Query / SWR	Server state (API data)	Caching, sync, revalidation	useQuery(...)

Choose the right tool based on project size and complexity. For most Next.js apps, Zustand or React Query works great!

6 Monitoring & Analytics

- Use Vercel Analytics for traffic insights.
- Use Sentry for error tracking.
- Monitor Core Web Vitals (LCP, FID, CLS).
- Set up logging for API failures and critical errors.

Tools:

- Vercel Analytics
- Sentry
- LogRocket
- Google Analytics

7 Final Takeaways

- ✓ Use a clean and scalable project structure.
- ✓ Prefer Server Components for better performance.
- ✓ Use environment variables and proper API handling.
- ✓ Implement error handling and monitoring.
- ✓ Follow best practices and keep learning!

"Build, Deploy, Learn, Repeat!"

Page 15 : Conclusion, Roadmap & Resources

1 What We Covered ?

- ✓ Introduction & why Next.js
- ✓ Folder structure & project setup
- ✓ Routing (Pages & App Router)
- ✓ Components, Layouts and Reusability
- ✓ Data Fetching (SSR, SSG, ISR, CSR).
- ✓ API Routes and Backend Integration
- ✓ Styling, Images and Optimization
- ✓ Authentication & Middleware
- ✓ Performance, Caching and SEO
- ✓ Deployment and CI/CD
- ✓ Real-world project structure
- ✓ Best practices and common issues

"From
zero to
deployment
— you now
have a strong
foundation
in Next.js!"



2 Next.js Learning Roadmap

1. Foundations (Week 1)

- Learn React basics (JSX, hooks, state)
- Setup Next.js project
- Understand routing & components



2. Intermediate (Week 2-3)

- Data fetching (SSR, SSG, ISR, CSR)
- API routes & environment variables
- Styling (Tailwind/CSS Modules)
- Authentication (NextAuth.js)



3. Advanced (Week 4-5)

- Middleware & security
- Performance optimization
- SEO & metadata
- Integrate with databases (Postgres / MongoDB)
- Build and deploy real-world projects



4. Real-World Practice (Ongoing)

- Clone existing apps (e.g., Notion, Dev.to)
- Build your own side project
- Work on open source
- Keep learning new features (RSC, Server Actions, etc.)

3 Recommended Resources

Type	Resource	Link / Platform
Official Docs	Next.js Documentation	https://nextjs.org/docs
Tutorial	Next.js Crash Course	YouTube (Net Ninja, Codevolution)
Practice	Build Real Projects	https://github.com/vercel/nextjs-examples
Community	Next.js Discord	https://nextjs.org/discord
Advanced	Vercel Blog & Updates	https://vercel.com/blog
Learn by Doing	Build & Deploy Projects	Your Own Ideas! 🚀

4 Project Ideas to Practice

- Personal Portfolio - Showcase your work and blogs.
- Blog Platform - Markdown support with SSG/ISR.
- E-commerce Store - Products, cart, payment integration.
- SaaS Dashboard - Authentication, protected routes.
- Real-time Chat App - Using WebSockets or Pusher.
- AI-powered App - Integrate with OpenAI API.
- Job Board - Listings with search and filters.
- URL Shortener - Simple and great for learning.
- Twitter Clone - Authentication + real-time feed.
- Video Streaming App - Use external APIs (YouTube, etc.).

Build
projects to
turn knowledge
into real
skills! 💡

5 Best Practices Checklist

- ✓ Use App Router (latest) for new projects.
- ✓ Keep components small and reusable.
- ✓ Use TypeScript for better development experience.
- ✓ Optimize images using next/image.
- ✓ Use environment variables for sensitive data.
- ✓ Follow proper folder structure.
- ✓ Implement error handling (error.tsx, not-found.tsx).
- ✓ Use metadata for SEO.
- ✓ Enable caching and revalidation where possible.
- ✓ Test thoroughly before deployment.
- ✓ Keep dependencies updated.
- ✓ Follow security best practices (auth, input validation, etc.).

Good
habits lead
to scalable
and maintainable
apps!

6 Common Pitfalls to Avoid

- ✗ Using Next.js for very small/static sites (overkill).
- ✗ Not handling loading and error states.
- ✗ Ignoring SEO and metadata.
- ✗ Exposing API keys in frontend code.
- ✗ Not optimizing images and assets.
- ✗ Large components and poor folder structure.
- ✗ Skipping error handling and logging.
- ✗ Not testing on production build (npm run build).
- ✗ Ignoring performance issues (large bundles, unused code).
- ✗ Not keeping up with new Next.js features.

Learn from
mistakes,
build better
next time!



7 Final Thoughts

- Next.js is a powerful and evolving framework.
- It gives you the flexibility to build any type of web application.
- Focus on building, breaking, fixing and improving.
- Keep exploring new features (RSC, Server Actions, Turbopack, etc.).
- Join the community, read blogs, and stay updated.
- Most importantly — keep building! 🚀

Code Today,
A Better Tomorrow!

